SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

SIMATS Engineering

Department of Computer Science and Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

ASSIGNMENT 2 – CO2

Brute-Force and Exhaustive-Search Techniques: Design, Experiment and Redesign

Student name	Register number	Section	Submission date

Report layout. Each question follows the order of the "Evidence Expected" list in the assignment: prediction before experiment, design, observed evidence, comparison of prediction with observation, and a decision. Operation counts are the primary evidence; execution times are secondary and come from a single Python 3 process on one machine, so only their ratios and growth shape should be trusted. All numbers in this report are produced by *solution_code.py* and stored in *raw_data*.

Q1 – Sorting Under Different Data Conditions (6 Marks)

1.1 Prediction before implementation (n = 1000)

Dispatch priority scores are sorted in ascending order. Let n be the number of orders and I the number of inversions (pairs that are out of order). The reasoning behind each prediction is the structure of the inner loop of each algorithm.

Algorithm	Comparisons predicted	Swaps / shifts predicted	Algorithmic reason
Selection Sort	Always $n(n-1)/2$ = 499,500, for every input order	At most $n-1$ swaps (one per outer pass)	The inner loop must scan the whole unsorted part to be sure of the minimum. It cannot learn from existing order, so it has no best case.
Bubble Sort (with early-exit flag)	$n-1$ = 999 if sorted; about $n^2/2$ if random or reversed	Exactly I swaps (each swap removes one inversion)	One pass stops the sort only if it makes no swap. A small element near the end moves only one place left per pass, so a single late arrival can force about n passes.
Insertion Sort	$n-1+I$ (999 if sorted, about $n^2/4$ if random, $n(n-1)/2$ if reversed)	Exactly I shifts	Each element is compared only until its place is found. The work is proportional to the disorder, so sorted or nearly sorted data is cheap.

Prediction summary. Selection Sort will be insensitive to the data (same comparisons everywhere). Bubble and Insertion Sort will both be fast on sorted data and slow on random or reversed data. Insertion Sort should beat Bubble Sort on every nearly sorted input because it performs the same number of data movements (I) with far fewer comparisons.

1.2 Designed datasets (n = 1000 each)

Dataset	Construction	Purpose
D1 Random	1000 distinct scores in random order	Contrasting dataset 1: typical disorder, about $n^2/4$ inversions
D2 Sorted	Scores already in ascending order	Contrasting dataset 2: zero disorder (best case for adaptive algorithms)

Dataset	Construction	Purpose
D3 Edge case	Sorted list of 999 orders with one new top-priority order (smallest score) appended at the end	Exposes the “turtle” weakness: only one inversion-heavy element, but it sits at the far end
D4 Fulfilment scenario	950 correctly ordered orders followed by 50 new random orders (95% / 5%)	Mirrors the 95% already-ordered report in the design decision
D5 Reverse	Strictly descending scores	Worst case, maximum I = n(n−1)/2
D6 All equal	Every score identical	Duplicates; checks that no algorithm does unnecessary movement

D1, D2 and D3 are the two contrasting datasets and the edge case required by the question. D4, D5 and D6 are supplementary datasets that test the specific business condition and the extreme cases. Every sorted result was checked against Python’s built-in sort.

1.3 Observed operation counts

Dataset	Selection comparisons / swaps	Bubble comparisons / swaps	Insertion comparisons / shifts
D1 Random order	499,500 / 992	496,497 / 249,484	250,479 / 249,484
D2 Already sorted	499,500 / 0	999 / 0	999 / 0
D3 Edge: sorted + 1 top-priority order at end	499,500 / 999	499,500 / 999	1,997 / 999
D4 Scenario: 95% sorted + 5% new batch appended	499,500 / 978	498,324 / 22,134	23,133 / 22,134
D5 Reverse sorted	499,500 / 500	499,500 / 499,500	499,500 / 499,500
D6 All equal scores	499,500 / 0	999 / 0	999 / 0

Table 1.1 – Comparisons and swaps/shifts for each algorithm (n = 1000). The highlighted row is the business scenario.

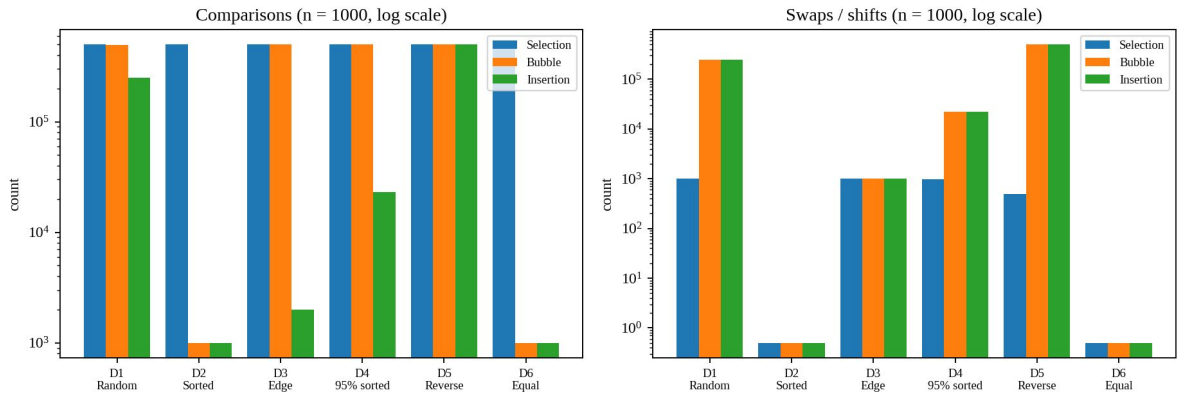


Figure 1.1 – Operation counts on a logarithmic scale.

Dataset	Selection (ms)	Bubble (ms)	Insertion (ms)
D1 Random order	23.762	47.516	26.454
D2 Already sorted	22.711	0.057	0.088
D3 Edge: sorted + 1 top-priority order at end	23.33	26.89	0.18
D4 Scenario: 95% sorted + 5% new batch appended	23.98	28.828	2.227
D5 Reverse sorted	23.819	57.316	44.93

Dataset	Selection (ms)	Bubble (ms)	Insertion (ms)
D6 All equal scores	23.131	0.057	0.159

Table 1.2 – Execution time (secondary evidence; best of 3 runs; includes counter overhead).

Scaling check on random data

n	Selection comparisons	Bubble comparisons	Insertion comparisons	Insertion shifts
250	31,125	30,915	15,368	15,123
500	124,750	124,735	60,608	60,116
1000	499,500	498,972	248,108	247,115
2000	1,999,000	1,994,905	981,485	979,491

Table 1.3 – Doubling n multiplies every count by about 4, confirming $\Theta(n^2)$ behaviour on random data.

1.4 Prediction versus observation

- **Selection Sort:** 499,500 comparisons on all six datasets, exactly n(n−1)/2. The prediction held, including on sorted data (D2), where it does 500 times more comparisons than Bubble or Insertion Sort.
- **Sorted data (D2):** Bubble and Insertion Sort both needed 999 comparisons and zero movements, as predicted.
- **Random data (D1):** Insertion Sort used 250,479 comparisons and 249,484 shifts, about half the comparisons of Bubble Sort (496,497). Both made 249,484 data movements, equal to the inversion count, which confirms that movements = I.
- **Edge case (D3), the main surprise:** only one order is out of place, yet Bubble Sort needed 499,500 comparisons, the same as Selection Sort, because the late arrival moves one position per pass. Insertion Sort handled it in 1,997 comparisons by walking that one element left in a single pass. This is the strongest evidence that Bubble Sort's early-exit flag does not make it adaptive in general.
- **Scenario (D4):** the data is 95% in order, yet Bubble Sort still performed 498,324 comparisons (almost the random-data cost) because some of the 50 new orders carry small scores that travel left slowly. Insertion Sort needed 23,133 comparisons, about 21.5 times fewer than Selection or Bubble Sort.
- **Reverse (D5):** all three reached their worst case, 499,500 comparisons; Bubble and Insertion Sort also made 499,500 movements. Selection Sort used only 500 swaps.

One prediction needed refinement: I expected Bubble Sort to benefit noticeably from the early-exit flag on "nearly sorted" data. The observations show the benefit depends on where the disorder sits, not on how little of it there is.

1.5 Design decision: 95% of orders already in priority order

To test the decision beyond one batch size, n was fixed at 1000 and the size of the appended batch was varied.

New batch	Share of n	Selection comparisons	Bubble comparisons	Insertion comparisons	Insertion shifts
1	0.1%	499,500	117,999	1,124	125
10	1%	499,500	486,134	6,573	5,574
50	5%	499,500	499,310	30,427	29,428
100	10%	499,500	498,225	49,846	48,847
250	25%	499,500	497,289	109,122	108,123

New batch	Share of n	Selection comparisons	Bubble comparisons	Insertion comparisons	Insertion shifts
500	50%	499,500	498,720	193,286	192,288

Table 1.4 – Effect of the size of the appended batch (sorted prefix + random new orders).

Recommendation: Insertion Sort. The defence rests on the evidence:

1. Its comparison count is n−1+I, so cost follows the disorder. For the 95%/5% scenario this is 23,133 comparisons, against 498,324 for Bubble Sort and 499,500 for Selection Sort, a reduction of more than 95%. Even with a batch of 250 new orders (25%) it stays about 4.5 times cheaper than the alternatives, and it is never worse than Bubble Sort.
2. It is robust to the awkward edge case. A single urgent order arriving last costs Insertion Sort 1,997 comparisons but costs Bubble Sort 499,500 (D3).
3. It is stable, so orders with equal priority scores keep their arrival order, which matters for fair dispatch. Selection Sort (swap-based) is not stable.
4. Its one weakness, element shifting, is small here: 22,134 shifts in the scenario, identical to Bubble Sort's swaps.

Where Selection Sort would be better: only if writing a record is far more expensive than comparing (it performs at most n−1 swaps; 978 in D4), for example if records were stored on flash with limited write cycles. **Limitation of the recommendation:** if the fulfilment centre sorts a very large file where each batch is a large fraction of the file, Insertion Sort still degrades to $n^2/4$. In that case sorting only the small new batch and merging it with the existing ordered list would be a better strategy, which is beyond the three algorithms compared here.

Q2 – When Is Sequential Search an Appropriate Engineering Choice? (4 Marks)

2.1 Experiment design

Each record is (Tool ID, Technician ID, Issue Time, Work Bay) and the list is unordered because tools are issued and returned continuously. Sequential Search compares the requested Tool ID with each record in turn and counts every Tool-ID comparison. The following experiments were run for list sizes n = 100, 1,000, 10,000 and 100,000:

- successful search with the key in the first, middle and last position;
- unsuccessful search (Tool ID not in the list);
- average over 2,000 randomly chosen existing Tool IDs (compared with the theoretical (n+1)/2).

Prediction: first position = 1 comparison; middle ≈ n/2; last = n; unsuccessful = n; average successful ≈ (n+1)/2. The algorithm has no way to rule out a record without examining it, so these values follow from the loop itself.

2.2 Comparison-count evidence

List size n	First	Middle	Last	Unsuccessful	Average (2,000 keys)	Theory (n+1)/2	Worst-case time (ms)
100	1	51	100	100	50.5	50.5	<0.01
1,000	1	501	1,000	1,000	500.5	500.5	0.05
10,000	1	5,001	10,000	10,000	5,039.5	5,000.5	0.51
100,000	1	50,001	100,000	100,000	49,960.1	50,000.5	6.3

Table 2.1 – Comparisons required by Sequential Search.

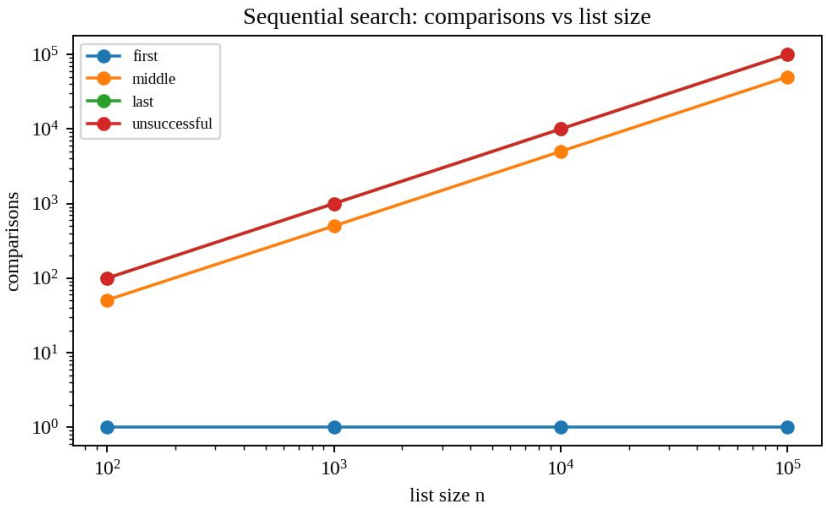


Figure 2.1 – Comparisons grow linearly with list size (parallel lines on log-log axes, slope 1).

Findings. The cost is exactly proportional to n: a ten-fold larger list needs ten times the comparisons for every scenario. The key's position decides the cost within one list (1 versus n), and an unsuccessful search is the most expensive, because the full list must be scanned before "not found" can be reported. Measured averages (50.5, 500.5, 5,039.5, 49,960.1) agree with (n+1)/2 within about 1%, the small differences being sampling noise.

2.3 Operating conditions and the point at which to stop recommending it

Measured throughput in this environment is about 13.6 million comparisons per second. Two acceptability criteria were defined: (i) a single worst-case lookup must finish within 50 ms, and (ii) the lookup workload must use no more than 10% of one processor core (leaving room for the rest of the system). With n comparisons per lookup in the worst case, criterion (i) allows up to n ≈ 681,452 records, and criterion (ii) requires lookups per second × n ≤ 1,362,905.

Active records n	Worst-case lookup latency (ms)	Maximum lookups per second (10% core)
500	0.04	2,726
2,000	0.15	681
10,000	0.73	136
50,000	3.67	27
100,000	7.34	14
500,000	36.69	3

Table 2.2 – Operating boundary derived from measured throughput (worst case: n comparisons per lookup).

Sequential Search is a reasonable choice when: the list is small enough that a full scan is far below the latency target, lookups are infrequent relative to that capacity (the product n × lookup-rate is what matters, not either alone), the list is unordered and changes constantly (so sorting or indexing has a maintenance cost on every issue and return), and simplicity and certainty of correctness are valued. A list of a few hundred tools queried a few times a minute is far inside the boundary: lookups cost microseconds and no additional data structure is needed.

I would stop recommending it when: n × lookup-rate approaches the boundary in Table 2.2, when each comparison becomes expensive (for example each record read from a database or across a network, which makes a comparison 1,000 times or more costlier than in memory), or when lookups must meet a tight guaranteed response time during peaks.

2.4 Changed condition: 20 maintenance facilities

Scenario (assumed figures)	Active records n	Lookups per second	Worst-case latency (ms)	Lookup load, % of one core
One hangar	500	0.1	0.04	0.0004
20 facilities (assumed)	10,000	20	0.73	1.47
20 facilities, 5x growth + peak	50,000	100	3.67	36.69

Table 2.3 – Re-evaluation. The workload figures are illustrative assumptions, labelled as such.

Re-evaluated recommendation. The evidence gives a nuanced answer. With 20 facilities pooled into one in-memory list of about 10,000 records and 20 lookups per second, the load is about 1.5% of one core and the worst lookup takes under 1 ms, so Sequential Search would still technically work. However, the system is now about 7 times from the lookup-rate limit at that size (136 lookups per second at 10% load), and a five-fold growth in records together with five-fold higher peak demand (50,000 records, 100 lookups per second) pushes the load to roughly 37%, which breaks the 10% criterion. Because the cost grows linearly with both records and requests, there is no margin for further growth, and a distributed 20-site system will likely read its records from a shared store, where each comparison is far more expensive. **I therefore withdraw the recommendation for the multi-site system.** The recommended replacement is an index keyed by Tool ID (a hash table), updated at each issue and return; since each Tool ID is held by at most one technician at a time, a lookup then needs an expected constant number of comparisons regardless of n. Sequential Search stays appropriate for a single hangar or for small, low-traffic lists.

Q3 – Engineer the Behaviour of Brute-Force String Matching (6 Marks)

3.1 Algorithm and counting rule

Brute-Force String Matching tries every alignment i = 0 … n−m of the pattern (length m) against the text (length n). At each alignment characters are compared left to right until the first mismatch or until all m match. Every individual character comparison is counted. The work at one alignment is therefore the length of the matching prefix plus one (if a mismatch occurs).

3.2 Test design, predictions and results (n = 10,000; m = 50)

Test	Text and pattern	Prediction made before running and reason	Predicted	Observed
T1 little work	Random lowercase text; pattern starts with “#”, a character that never occurs in the text	The first comparison at every alignment fails, so each alignment costs 1 comparison: n−m+1 = 9,951.	9,951	9,951
T2 engineered	Text = 10,000 letters “A”; pattern = 49 “A” followed by “B”	Every alignment matches 49 characters and fails at the 50th, so every alignment costs m comparisons: m(n−m+1) = 497,550.	497,550	497,550
T3 own choice	Random binary text (0/1); random binary pattern of length 50	A random mismatch occurs with probability 1/2 at each position, so the expected cost per alignment is Σ(1/2)^k = about 2: about 2(n−m+1) = 19,902.	19,902	19,956

Table 3.1 – Predicted versus observed character comparisons.

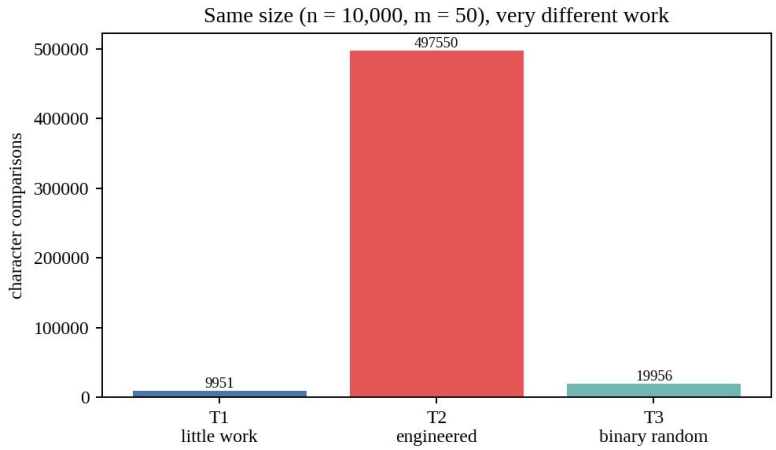


Figure 3.1 – Identical n and m, yet a 50-fold spread in character comparisons.

T1 and T2 matched their predictions exactly. T3 differed from its expected value by 54 comparisons (0.3%), which is the sampling variation expected for random text. The expected-cost formula was tested further across alphabet sizes:

Alphabet size	Predicted (n−m+1)(1−p^m)/(1−p), p = 1/σ	Observed
2	19,902	20,052
4	13,268	13,254
26	10,349	10,384

Table 3.2 – Smaller alphabets cause more partial matches and therefore more comparisons.

Pattern length m	Predicted m(n−m+1)	Observed (all-A text, pattern A…AB)
5	49,980	49,980
10	99,910	99,910
25	249,400	249,400
50	497,550	497,550
100	990,100	990,100
200	1,960,200	1,960,200

Table 3.3 – In the engineered case, work grows linearly with the pattern length.

3.3 Inquiry

Why one input requires substantially more work. In T1 each alignment is rejected after one comparison, so the pattern length plays no role and the work is about n. In T2 every alignment is rejected only at the last pattern character, so all earlier comparisons are wasted and the work is about n × m (497,550 against 9,951, a factor of 50 equal to m). The algorithm never uses what it learned at one alignment to skip the next; each alignment starts again from scratch.

Characteristics that create repeated comparisons. (1) Repetitive text with long runs of the same characters; (2) a pattern whose prefix occurs frequently in the text, so alignments survive many comparisons before failing; (3) a mismatch located near the end of the pattern instead of its start; (4) a small alphabet, where chance matches are likely (T3: about 2 comparisons per alignment for binary data against about 1.04 for 26 letters). For security logs this matters, because log lines share long identical prefixes (timestamps, host names, “Failed password for …”), so real logs lie between T1 and T2 and a signature that shares its prefix with common lines will behave worse.

What this reveals about evaluating an algorithm using only input size. All three tests had the same text length and pattern length, yet the work ranged from 9,951 to 497,550 comparisons. Input size alone cannot predict running time. The structure and content of the input matter, so an honest evaluation states best, average and worst cases (here roughly n, a few times n, and n × m) and tests on inputs engineered for each, rather than quoting one figure based on n and m.

Q4 – Closest Pair and the Cost of Examining Every Possibility (6 Marks)

4.1 Design and derivation of the pair count

The closest pair of n drones can be found only by comparing the distances of candidate pairs. The candidates are all unordered pairs {Di, Dj} with i ≠ j. Rules used to avoid duplicates:

- a drone is never paired with itself (i ≠ j);
- the pair (Di, Dj) is the same as (Dj, Di), so only pairs with i < j are examined: the inner loop starts at j = i+1.

Derivation. Drone 1 is paired with n−1 later drones, drone 2 with n−2, and so on, down to drone n−1 with 1 drone and drone n with none:

$$(n-1) + (n-2) + \ldots + 2 + 1 = n(n-1)/2 = C(n, 2)$$

This equals the number of ways to choose 2 drones from n. For example n = 5 gives 10 pairs and n = 100 gives 4,950. The count grows as $n^2/2$: doubling n multiplies the pairs by about 4.

```
ALGORITHM BruteForceClosestPair(D[1..n])
    best ← ∞; pairs ← 0
    for i ← 1 to n−1 do
        for j ← i+1 to n do
            pairs ← pairs + 1
            d ← (xi − xj)² + (yi − yj)²          // squared distance: no square root needed to compare
            if d < best then best ← d; (a, b) ← (i, j)
    return (a, b), √best
```

Squared distances are compared, which gives the same minimum without computing a square root. Therefore the number of distance calculations equals the number of pairs examined. A hand-checkable test (points (0,0), (10,10), (3,4), (11,11), (20,0)) examined 10 pairs, as the formula gives, and returned the pair (10,10)–(11,11) at distance 1.414, which is correct.

4.2 Experimental results

Drones n	Pairs examined	n(n−1)/2	Distance calculations	Time (s)	Time ratio to previous n
100	4,950	4,950	4,950	0.0005	–
200	19,900	19,900	19,900	0.0020	4.34
400	79,800	79,800	79,800	0.0085	4.21
800	319,600	319,600	319,600	0.0379	4.47
1,600	1,279,200	1,279,200	1,279,200	0.1565	4.13
3,200	5,118,400	5,118,400	5,118,400	0.5707	3.65

Table 4.1 – Observed pairs equal n(n−1)/2 exactly at every size; time grows by about 4 when n doubles.

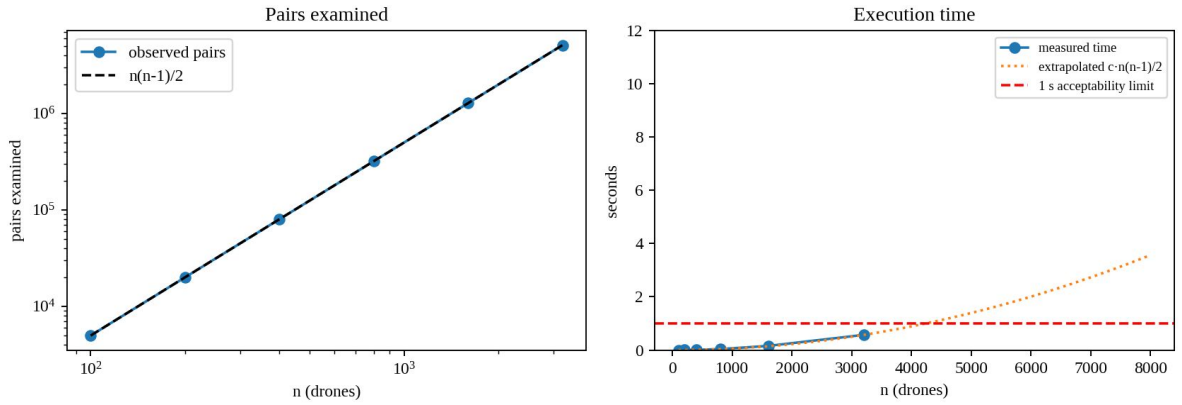


Figure 4.1 – Left: observed pairs lie exactly on n(n−1)/2. Right: measured time and extrapolation against the 1 s limit.

The theoretical and experimental pair counts are identical, and execution time follows the quadratic prediction (each doubling of n multiplies time by about 4).

4.3 Scalability inquiry

Acceptability criterion. I define an acceptable response time as 1 second per monitoring cycle in this environment, assuming drone positions are refreshed about once per second. The measured cost is 111 ns per pair (from n = 3,200), so the pairs that fit in 1 second number about 9.0 million.

Practical limit. Solving n(n−1)/2 = 9.0 million gives n ≈ 4,235 drones: the brute-force approach is comfortable up to about 3,200 drones (0.57 s measured) and becomes impractical beyond about 4,200. Extrapolation gives about 5.6 s for 10,000 drones and about 557 s (more than 9 minutes) for 100,000 drones. A cycle limit of 10 s would extend the limit only to about 13,393 drones: a 10-fold relaxation of the time limit buys only a √10 ≈ 3.2-fold gain in fleet size.

Origin of the inefficiency. The cost comes from the double loop: every pair is examined regardless of position, although drones that are far apart cannot possibly be the closest pair. Each pair is evaluated independently and no information is reused: after a close pair has been found, nothing is learned that rules out other pairs. The work is dominated by the very large number of obviously irrelevant, distant pairs.

What a redesigned strategy must do. It must (1) avoid examining distant pairs, by exploiting spatial locality (for example by ordering or partitioning the points so that only neighbours are compared); (2) use the best distance found so far as a cut-off to discard candidates that cannot improve it; and (3) reuse information from earlier comparisons or from a preprocessing step, accepting an upfront cost (such as ordering the points) in return for far fewer pair examinations.

Q5 – Derive a Convex Hull Algorithm from a Geometric Principle (7 Marks)

5.1 From the geometric principle to a computational procedure

The given principle is: *a line joining two candidate points may form part of the outer boundary if all remaining points lie on one side of that line.* It is converted into computation in five steps.

Step 1: two points define a line. For P = (x1, y1) and Q = (x2, y2) the line is a·x + b·y = c, with a = y2 − y1, b = x1 − x2 and c = x1·y2 − y1·x2.

Step 2: position of another point. For a point R = (xr, yr) compute s(R) = a·xr + b·yr − c. If s(R) > 0, R is on one side of the line; if s(R) < 0, it is on the other side; if s(R) = 0, it lies on

the line. For the directed line from P to Q, s(R) < 0 means R is on the left (counter-clockwise side), and s(R) > 0 means it is on the right.

Step 3: examine the remaining points. For every ordered pair (P, Q), evaluate s(R) for every other point R. The test stops early as soon as one point is found on the wrong side.

Step 4: hull-edge condition. The directed segment P→Q is accepted as a hull edge if s(R) ≤ 0 for every other point R, and every point with s(R) = 0 lies between P and Q on the segment (so that only the extreme points of collinear boundary points are kept).

Step 5: constructing the boundary. Accepted directed edges all have the interior on their left, so they form a counter-clockwise loop. A successor table (next[P] = Q) is built and the boundary is traced from the lowest-leftmost point until it returns to the start. Duplicate points are removed first; if both P→Q and Q→P are accepted, all points are collinear and the hull is the two extreme points.

```
ALGORITHM BruteForceHull(S)
    S ← distinct points of S;  tests ← 0;  E ← empty set of edges
    if |S| ≤ 2 then return S
    for each point P in S do
        for each point Q in S, Q ≠ P do
            a ← Qy − Py;  b ← Px − Qx;  c ← Px·Qy − Py·Qx
            accepted ← true
            for each point R in S, R ≠ P, R ≠ Q do
                s ← a·Rx + b·Ry − c;  tests ← tests + 1
                if s > 0 or (s = 0 and R is not between P and Q) then
                    accepted ← false;  break
            if accepted then add edge (P → Q) to E
    if E contains an edge and its reverse then return the two extreme points
    next[P] ← Q for every (P → Q) in E
    hull ← [lowest-leftmost point];  v ← next[hull[1]]
    while v ≠ hull[1] do append v to hull;  v ← next[v]
    return hull
```

Integer coordinates are used so that the sign of s(R) is exact (no floating-point error when a point lies exactly on a line).

5.2 Test configurations and visualisation

Set 1: 30 random points, some forming the boundary and the rest interior. **Set 2 (designed to challenge the algorithm):** a 5 × 5 integer grid with duplicate points added. It contains many collinear triples, collinear points in the middle of every boundary edge (which must not become vertices) and repeated points. Additional stress configurations were also run. The result of every test was compared with an independent reference (a monotone-chain hull used only for checking).

Test configuration	Distinct points	Hull vertices found	Side tests	Matches reference
Set 1: 30 random points (boundary + interior)	30	9	4,807	Yes
Set 2: 5x5 grid + duplicates (collinear boundary points)	25	4	2,674	Yes
Triangle + 3 interior points	6	3	61	Yes
All points collinear	8	2	96	Yes
Parabola y=x^2 (all points on hull)	13	13	561	Yes
Two points only	2	2	0	Yes

Table 5.1 – All test configurations. Set 2 correctly returns four vertices only, despite 16 boundary grid points.

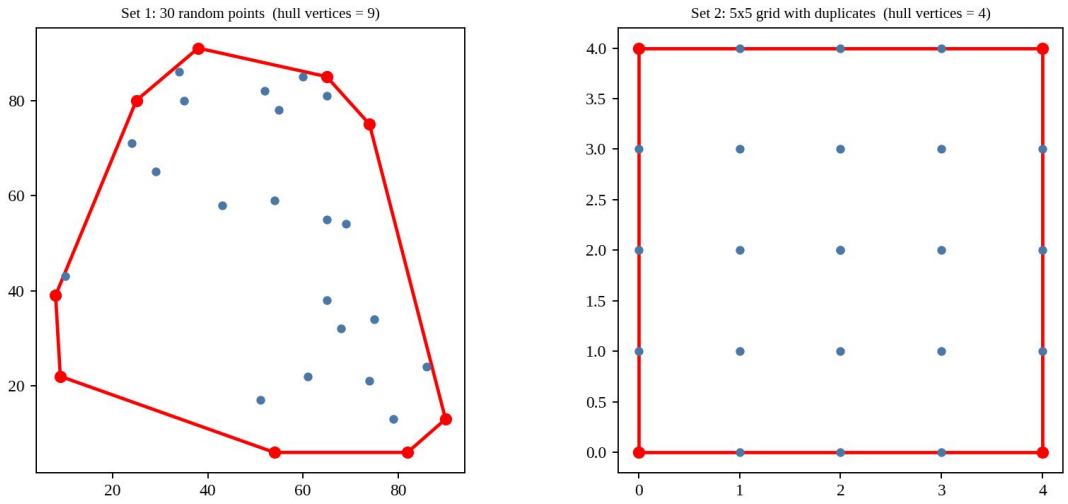


Figure 5.1 – Computed boundary (red) for the two required test sets.

In Set 2 the 12 boundary grid points that are not corners lie exactly on a boundary line (s = 0). Any pair that includes one of them as an end point is rejected, because another collinear point falls outside its segment, so only the corner-to-corner pairs are accepted and the vertices (0,0), (4,0), (4,4), (0,4) are returned. The all-collinear set returned the two extreme points, and the parabola returned all 13 points as vertices, as expected because every point is extreme.

5.3 Computational growth

n	Hull vertices	Side tests (no early exit)	n(n−1)(n−2)	Side tests (early exit)	Time with early exit (s)
10	4	720	720	241	0.000
20	8	6,840	6,840	1,616	0.000
40	9	59,280	59,280	9,866	0.002
80	11	492,960	492,960	67,254	0.016
160	20	4,019,520	4,019,520	504,090	0.107
320	14	not run	32,461,440	3,478,278	1.044
640	18	not run	260,916,480	24,454,330	5.006

Table 5.2 – Growth of work on random points.

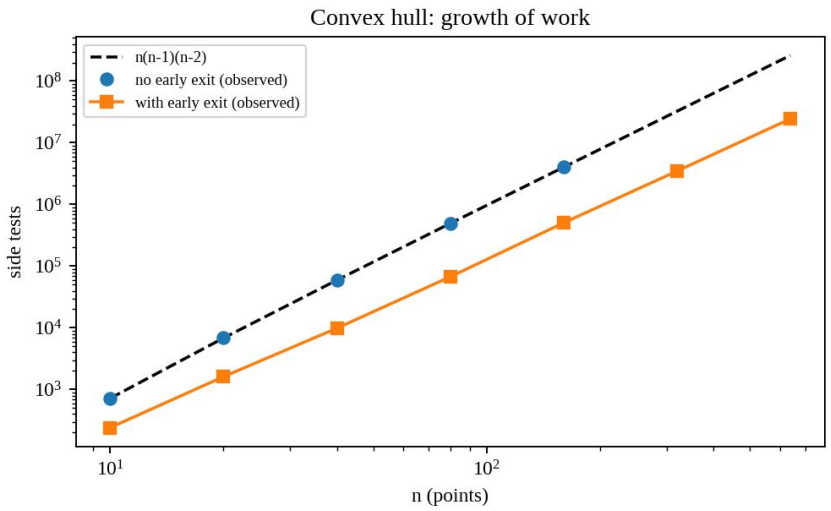


Figure 5.2 – Without early exit the work is exactly n(n−1)(n−2); early exit lowers it by a constant factor.

Without early exit the algorithm performs exactly n(n−1)(n−2) side tests, matching the count for n up to 160 (for example 4,019,520 at n = 160). Each of the n(n−1) ordered pairs is checked against n−2 other points, so the growth is $\Theta(n^3)$: doubling n multiplies the work by about 8. Early exit reduces the constant (about 8 times fewer tests at n = 160) but not the order of growth: the observed tests rise by a factor of about 7 per doubling (roughly n^2.8 over the tested range), and for points on a convex curve the early exit saves little (720,322 tests at n = 160 against 4,019,520). At n = 640 the early-exit version already takes about 5 s.

5.4 Correctness inquiry: why an accepted edge belongs to the boundary

Soundness (accepted edges are on the boundary). Suppose P→Q passes the test. Then every point of the set lies in the closed half-plane H on the left of line PQ. A half-plane is convex, so the smallest convex set containing all the points, the convex hull, also lies inside H. The points P and Q belong to the set and lie on the bounding line of H, so the line touches the hull and has the whole hull on one side: it is a supporting line of the hull. The hull's intersection with a supporting line is part of its boundary, so segment PQ lies on the boundary. No part of the hull can lie on the other side of that line, so PQ is a boundary edge, or part of one.

Completeness (no boundary edge is missed). Every edge of the hull lies on a supporting line, which has all points on one side. Its endpoints are two points of the set, so the pair passes the test and is accepted. The collinear rule makes sure that, when several points lie along one boundary edge, only the two extreme ones form an accepted pair (a pair with a collinear point outside its segment is rejected), so intermediate points are not reported as vertices.

Why interior points never create edges. A line through a point strictly inside the hull always has points of the set strictly on both sides of it (one with s > 0 and one with s < 0), so any pair that contains an interior point fails the test. The correctness therefore follows from the geometry of supporting lines rather than from the program output.

Q6 – Exhaustive Search and Combinatorial Growth (8 Marks)

6.1 Design of the exhaustive search

- **Representation.** A candidate is a binary vector x = (x1, …, xn), where xi = 1 means package i is loaded. This is equivalent to an n-bit integer from 0 to 2^n-1, so each combination has a unique code.

- **Generation.** Depth-first assignment: at package i, first set xi = 1 and then xi = 0, recursing to package i+1. When i = n, one complete candidate has been produced, so all 2^n vectors are generated exactly once.
- **Feasibility check.** For a complete candidate, total weight Σ wi·xi must not exceed the payload limit W.
- **Evaluation.** Total value Σ vi·xi is computed for feasible candidates (running sums are passed down the recursion so each step costs O(1)).
- **Retaining the best.** Keep (best value, best vector); replace them only when a feasible candidate has a strictly greater value.

```
ALGORITHM ExhaustiveAirlift(w[1..n], v[1..n], W)
    bestValue ← −1;  bestVector ← none
    Generate(1, 0, 0, [ ])
    return bestValue, bestVector

PROCEDURE Generate(i, weight, value, x)
    if i = n+1 then                                  // complete candidate
        if weight ≤ W and value > bestValue then
            bestValue ← value;  bestVector ← copy of x
        return
    Generate(i+1, weight + w[i], value + v[i], x + [1])    // load package i
    Generate(i+1, weight, value, x + [0])                  // leave package i
```

6.2 Correctness check on a small instance

Five packages, payload limit W = 26: weights (12, 7, 11, 8, 9) and values (24, 13, 23, 15, 16) for P1–P5. There are 2^5 = 32 subsets, small enough to verify by hand. All of them are listed below (a subset is feasible if its weight is at most 26).

Subset	Weight	Value	Feasible	Subset	Weight	Value	Feasible
{∅}	0	0	Yes	{P5}	9	16	Yes
{P1}	12	24	Yes	{P1,P5}	21	40	Yes
{P2}	7	13	Yes	{P2,P5}	16	29	Yes
{P1,P2}	19	37	Yes	{P1,P2,P5}	28	53	No
{P3}	11	23	Yes	{P3,P5}	20	39	Yes
{P1,P3}	23	47	Yes	{P1,P3,P5}	32	63	No
{P2,P3}	18	36	Yes	{P2,P3,P5}	27	52	No
{P1,P2,P3}	30	60	No	{P1,P2,P3,P5}	39	76	No
{P4}	8	15	Yes	{P4,P5}	17	31	Yes
{P1,P4}	20	39	Yes	{P1,P4,P5}	29	55	No
{P2,P4}	15	28	Yes	{P2,P4,P5}	24	44	Yes
{P1,P2,P4}	27	52	No	{P1,P2,P4,P5}	36	68	No
{P3,P4}	19	38	Yes	{P3,P4,P5}	28	54	No
{P1,P3,P4}	31	62	No	{P1,P3,P4,P5}	40	78	No
{P2,P3,P4}	26	51	Yes	{P2,P3,P4,P5}	35	67	No
{P1,P2,P3,P4}	38	75	No	{P1,P2,P3,P4,P5}	47	91	No

Table 6.1 – Complete enumeration of the 32 subsets. 18 are feasible; the highlighted subset has the greatest value.

Checking by hand, the best feasible subset is {P2, P3, P4} with weight 7 + 11 + 8 = 26 and value 13 + 23 + 15 = 51. No other feasible subset exceeds 51: the next best are {P1, P3} with value 47 and {P2, P4, P5} with value 44. The program generated 32 candidates, found 18 feasible, and returned P2, P3, P4 with value 51, identical to the manual result. The result also agrees with an independent dynamic-programming check included in the test code.

6.3 Growth investigation

Prediction before experimentation. Each package has two states (loaded or not loaded), independent of the others, so the number of candidate combinations is $2 \times 2 \times \ldots \times 2 = 2^n$. Each additional package doubles the number of candidates and, since the work per candidate is constant, doubles the running time; adding 10 packages multiplies the work by $2^{10} = 1{,}024$.

n	Predicted candidates 2^n	Candidates generated	Feasible	Time (s)	Time ratio to previous row
10	1,024	1,024	311	0.0004	–
12	4,096	4,096	1,079	0.0016	4.11
14	16,384	16,384	4,174	0.0063	3.97
16	65,536	65,536	15,721	0.0253	4.01
18	262,144	262,144	59,960	0.1017	4.03
20	1,048,576	1,048,576	214,253	0.4023	3.95
22	4,194,304	4,194,304	829,309	1.7545	4.36

Table 6.2 – Observed candidates equal 2^n at every tested size (rows go up in steps of 2, so each row should be about 4 times the previous).

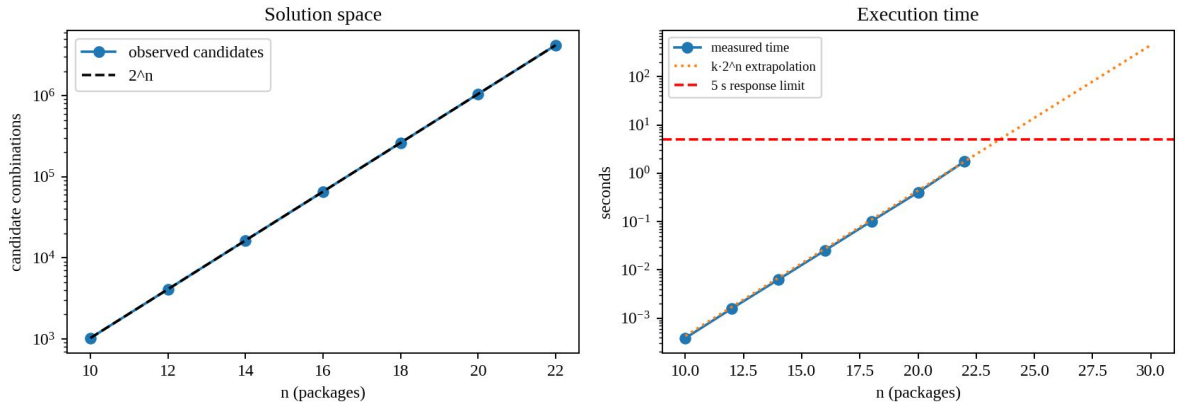


Figure 6.1 – Exponential growth of candidates (left) and time (right). Vertical axes are logarithmic.

The prediction is validated at seven sizes (n = 10 to 22): candidates equal exactly 2^n, and the time grows by about 4 for every two packages added (about 2 per package), as predicted. The correct optimum was returned at all seven sizes (each checked against dynamic programming).

6.4 Inquiry: why a few more packages cause a dramatic increase

The number of candidates is not added to but multiplied by 2 per extra package, because every existing combination can be extended in two ways (with or without the new package). This is exponential growth: the solution space is doubled again and again. From 22 to 32 packages, only 10 more (45% more packages), the number of candidates rises from 4.2 million to 4.3 billion (1,024 times). In contrast, a polynomial algorithm such as n^2 would grow from 484 to 1,024 operations (about 2.1 times) over the same range. The same increase from 22 to 40 packages (a factor of 262,144) would turn a 1.8 s computation into roughly 5 days. Exhaustive search spends

its effort in proportion to the size of the solution space, not the size of the input, and that space grows far faster than the input.

6.5 Deployment decision

Response-time limit chosen: 5 seconds. A loading plan for a relief flight is prepared before take-off and is reviewed by a planner, who can accept a short wait but not minutes, and 5 seconds leaves a safety margin if the planner's computer is slower. From the measured per-candidate cost (418 ns per candidate, at n = 22) the time model $t \approx k \times 2^n$ predicts about 3.5 s for 23 packages, 7.0 s for 24 packages and about 56 s for 27 packages.

Deployment boundary: 22 packages. At n = 22 the measured time is 1.75 s, which leaves a safety factor of about 2.9 against the 5 s limit on a slower machine. At n = 23 the predicted time (about 3.5 s) is within the limit on this machine but with little margin, and n = 24 (about 7 s) exceeds it. I would therefore deploy the plain exhaustive search only for loads of up to 22 packages, and for larger loads I would require a redesigned search (see Q7).

Q7 – Diagnose, Redesign and Validate (13 Marks)

Selected solution: the exhaustive airlift search from Q6. It shows the most severe limitation: a deployable size of only 22 packages in 5 seconds, because the work doubles with every added package.

A. Diagnose the computational bottleneck (2 Marks)

- **Where most effort is spent:** in generating and checking complete candidates. At n = 22 there are 4,194,304 candidates, and the search visits 8,388,607 tree nodes ($2^{n+1} - 1$), all of which are always visited.
- **Operation that grows:** the number of leaf candidates, doubling per package (1,024 at n = 10 and 4,194,304 at n = 22); the time grew from 0.39 ms to 1.75 s.
- **Unnecessary work:** only 829,309 of 4,194,304 candidates (19.8%) are feasible at n = 22, so 80.2% of the candidates are generated completely and then rejected at the leaf. At n = 10 the share of infeasible candidates was 69.6%. Feasible candidates with a clearly poor value are also fully evaluated although they cannot beat the best solution already known. Work is repeated in the sense that a partial selection that is already over the payload limit is extended to every one of its 2^(n−i) descendants, each of which must fail.
- **Characteristic of the original algorithm that causes the limitation:** it is blind. Feasibility is tested only at the leaves and the value is compared only at complete candidates, so information available in partial selections (current weight, current value, remaining capacity) is never used to cut off a branch.

B. Define the design objective (2 Marks)

The redesign must avoid examining candidates that cannot be (or contain) the optimum. Measurable objectives:

5. **Never extend a partial selection that already exceeds the payload limit** (removes the infeasible subtrees without generating their leaves).
6. **Never extend a partial selection whose best possible completion cannot beat the best solution found so far.**
7. **Quantitative targets:** at least 100 times fewer tree nodes than the $2^{n+1} - 1$ of the original at n = 20; node growth per added package below 1.3 (against 2 for the original); and an optimal value identical to exhaustive search and to the dynamic-programming check on every instance (no loss of correctness).

C. Design the revised computational strategy (4 Marks)

What changes. The search remains a depth-first construction of selections, but it now stops extending a partial selection as soon as it can prove that extending it is pointless.

- **Property exploited, rule 1 (feasibility):** weights are non-negative, so a partial selection that already exceeds W can never become feasible. The “load” branch is skipped if weight + w[i] > W, which removes all 2^(n−i−1) descendants.
- **Property exploited, rule 2 (optimistic bound):** relax the problem by allowing the remaining capacity to be filled with fractions of packages, taking packages in decreasing order of value per kilogram. Because a fraction can only help, the relaxed value is an upper bound on any real completion. If floor(bound) ≤ the best value already found, the whole subtree cannot hold a better solution and is discarded (values are integers, so a tie cannot improve the solution).

Table 7.3 – The pruning idea has little power when the bound is weak.

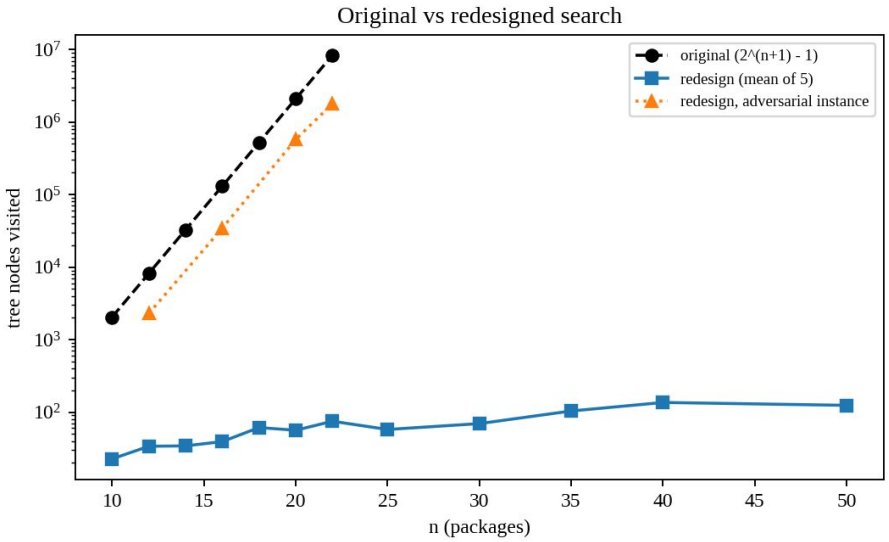


Figure 7.1 – Nodes visited by the original, the redesign on random instances, and the redesign on the adversarial instance (log scale).

F. Defend the final design (2 Marks)

- **Prediction supported?** Yes for predictions 1–4. At n = 20 the node count fell from 2,097,151 to 56.2 (37,316 times, far beyond the 100-times target), and the reduction rose to 112,147 times at n = 22. Node growth per added package was about 1.04 (22.6 nodes at n = 10, 124 at n = 50), far below 1.3. As predicted in prediction 3, time improved by less than nodes, though still enormously (n = 22: 1852.9 ms to 0.09 ms). On the adversarial instance the redesign visited only about 21–28% of the original nodes, confirming the limit predicted in prediction 4.
- **Magnitude of improvement:** on random instances the problem sizes the planner can handle moved from 22 packages to at least 50 packages in well under a millisecond, whereas the original would need an estimated 15 years for 50 packages.
- **New costs and trade-offs:** an O(n log n) sort, an O(n) bound calculation at each node, a more complex correctness argument, and data-dependent performance. The very large improvement is partly a property of the test data: with a payload of 40% of the total and varied value/weight ratios the greedy solution is almost optimal and the bound is tight. The improvement is a measurement on that class of instances, not a guarantee.
- **When the original may still be preferable:** for very small n (up to about 12 packages) where both finish instantly and the simpler code is easier to verify; when every feasible combination must be listed (for example to offer the planner alternatives), since the pruned search deliberately skips them; and as a trusted reference to test the optimised search, which is how it was used here.
- **Does the redesign sufficiently address the limitation?** It removes the limitation on typical data, but not in the worst case. On the adversarial instance the search still needs about 1.8 million nodes at n = 22 and grows by roughly 1.8 times per added package (576,293 nodes at n = 20 and 1,796,087 at n = 22), which is still exponential. For deployment, I would use the redesign with a time-out and keep a conservative guaranteed boundary of about 24 packages (the largest adversarial case that stays within 5 s, estimated from the 1.8-times growth per package), with larger loads accepted on the strength of the pruning evidence but monitored. A method with a guaranteed bound for all inputs would require a different technique (for example dynamic programming when weights are small integers), which is outside the scope of this redesign.

Summary of Findings

Question	Key evidence	Decision
Q1 Sorting	D4: Insertion 23,133 comparisons against 498,324 (Bubble) and 499,500 (Selection); D3 edge case exposes Bubble Sort	Recommend Insertion Sort
Q2 Sequential search	Comparisons = 1 … n, unsuccessful = n; average (n+1)/2	Fine for small, low-traffic lists; move to a hash index across 20 sites
Q3 String matching	Same n, m: 9,951 to 497,550 comparisons	Input size alone is insufficient; structure decides the cost
Q4 Closest pair	Pairs = n(n−1)/2 exactly; 1 s limit reached at about 4,200 drones	Brute force impractical above about 4,200 drones; need locality-based pruning
Q5 Convex hull	Work n(n−1)(n−2) without early exit; collinear and duplicate cases handled	Correctness proved by supporting-line argument
Q6 Exhaustive search	Candidates = 2^n; time about 2 times per package	Deploy plain search only to 22 packages (5 s limit)
Q7 Redesign	Nodes at n = 20: 2,097,151 to 56 (37,000 times fewer); adversarial case only 4 times fewer	Adopt pruned search; keep conservative boundary for worst case

Appendix – Submission Contents and Reproduction

- **solution_code.py:** source code for all implemented solutions (Q1–Q7). Run with "python3 solution_code.py"; it regenerates every table value, chart and data file.
- **raw_data (folder of CSV files):** raw experimental data for each table in this report (q1_… to q7_…).
- **charts (folder of PNG files):** graphs used in the analysis.
- **results.json:** all numbers used to build this report.

Random inputs use fixed seeds, so operation counts reproduce exactly. Execution times depend on the machine; re-running the script will give different times but the same counts, growth shapes and conclusions.